Software Architecture & System Design Document (SDD)

Project: QuickBite — Cloud-Native Distributed Food Delivery & Intelligent Dispatch Platform

> **Course Case Study**: Feng Chia University — Software Engineering (ASE) Practical Demonstration Material
> **Version**: v2.4 (Official Release)
> **Aligned Slide Modules**: Ch04 Object-Oriented System Modeling & Ch05 Software Architecture & Design
> **Standard Compliance**: Conforms to IEEE 1016-2009 Standard for Information Technology — Systems Design — Software Design Descriptions & Kruchten's 4+1 Architectural View Model

---

1. Architectural Overview & Kruchten's 4+1 Views

1.1 Architectural Style

QuickBite implements a **Microservices Architecture** integrated with an **Event-Driven Architecture (EDA)**.
The system leverages modern cloud-native design patterns to achieve high horizontal scalability, resilience, and strict domain separation:

Architecture Tier	Core Technology Stack	Key Engineering Responsibilities & Protocols
1. Client Tier	Mobile (iOS/Android) & Web SPA	Customer ordering UI, Merchant console, and Courier app over HTTPS / WSS
2. Cloud API Gateway	Kong / Envoy + OAuth 2.0 Auth	SSL termination, rate limiting, route management, and JWT validation
3. Domain Microservices	Order / Restaurant / Dispatch / Payment	Order lifecycle, menu & prep, courier matching engine, and Stripe/Bank payment (Spring Boot / Go)
4. Event Streaming Bus	Apache Kafka	High-throughput asynchronous event decoupling (`order.created` , `order.paid` , `order.prep_ready` , `courier.matched`)
5. Polyglot Persistence & Push	PostgreSQL / MongoDB / Redis / WebSocket	PostgreSQL (ACID orders), MongoDB (logs), Redis Cluster (GeoJSON & locks), and WebSocket push gateway

1.2 Kruchten's 4+1 View Model Alignment

Following the 4+1 View Model taught in **Slide 05**:

1. **Use Case View**: Defines the core business requirements and actor contracts driving the architecture (see SRS Figures 3.1 & 3.2).
2. **Logical View**: Encapsulates domain abstractions, inheritance hierarchies, and design patterns via the Domain Class Diagram.
3. **Process View**: Captures concurrency, asynchronous message streaming, and distributed state transitions via Sequence and State Machine diagrams.
4. **Development View**: Governs code organization, microservice boundaries, Maven/Gradle module dependencies, and OpenAPI contracts.
5. **Physical View**: Dictates container deployment topologies across Kubernetes (EKS/GKE) worker nodes and Multi-Availability-Zone (Multi-AZ) failovers.

---

2. Static Structural Modeling: Domain Class Model

2.1 Domain Class Diagram

In Object-Oriented Analysis and Design (OOAD), the domain class diagram serves as the structural backbone, formalizing domain entities, attributes, operations, and relationships.

The UML Domain Class Diagram below illustrates the entity structure of the QuickBite platform:

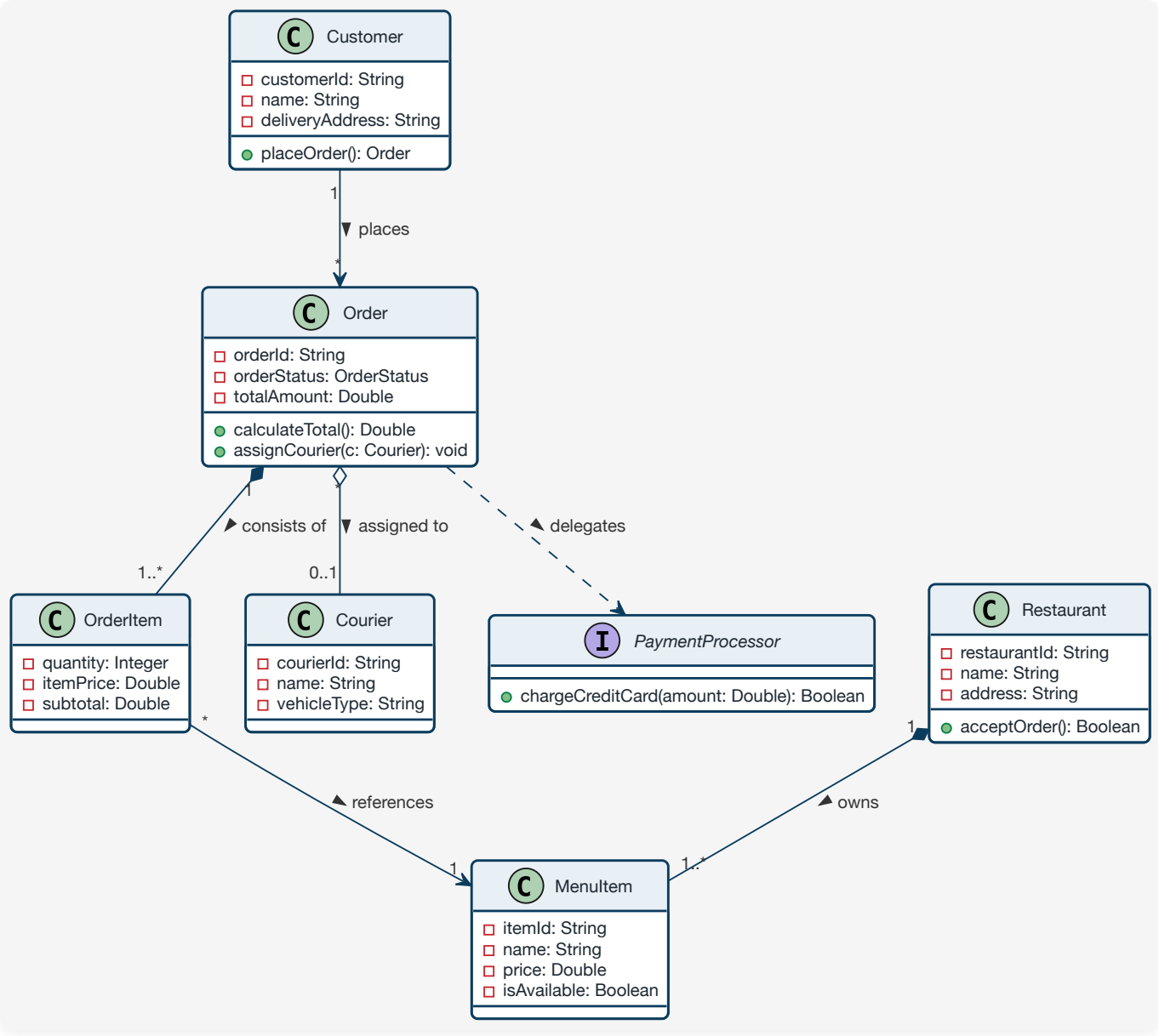


Figure 2.1 QuickBite Platform UML Domain Class Diagram

2.2 Class Anatomy & Relationship Semantics

1. Generalization / Inheritance (`User` Hierarchy)

- `User` serves as an abstract base class encapsulating shared identity attributes (`userId`, `name`, `phone`, `email`).
- `Customer` and `Courier` inherit from `User`:
  - `Customer` introduces address books and saved payment tokens.
  - `Courier` introduces vehicle attributes (`vehicleType`), real-time coordinates (`currentLocation`), performance ratings (`rating`), and shift availability (`status`).

2. Composition (`Order` *-- `OrderItem`)

- `Order` and `OrderItem` exhibit a solid diamond **Composition** relationship.

- **Lifecycle Co-dependence**: An `OrderItem` cannot exist independently of an `Order`. If an order is purged or archived, its constituent line items are cascade-purged.
- `OrderItem` captures a point-in-time price snapshot (`unitPrice`), insulating historical financial records from subsequent restaurant menu price modifications.

3. Aggregation (`Restaurant` o-- `MenuItem`)

- `Restaurant` maintains a 0..* collection of `MenuItem` objects via hollow diamond **Aggregation**.
- Menu items represent independent culinary entities. Even if a restaurant temporarily pauses operations or archives a menu category, historical item identifiers persist for inventory and analytical auditability.

2.3 GoF Design Patterns in Action

To enforce high cohesion, loose coupling, and the Open/Closed Principle (OCP), QuickBite incorporates three foundational GoF design patterns:

Pattern 1: Strategy Pattern — Intelligent Courier Dispatch

The matching engine alternates between optimization algorithms based on environmental conditions and regional traffic:

```
@startuml
interface DispatchStrategy <<interface>> {
  + matchCourier(order: Order): Courier
}
class NearestCourierStrategy {
  + matchCourier(order: Order): Courier
}
class RatingWeightedStrategy {
  + matchCourier(order: Order): Courier
}
DispatchStrategy <|.. NearestCourierStrategy
DispatchStrategy <|.. RatingWeightedStrategy
@enduml
```

```
public interface DispatchStrategy {
    Courier matchCourier(Order order, List<Courier> availableCouriers);
}

// Concrete Strategy A: Distance Minimization (Surge Traffic Mode)
public class NearestCourierStrategy implements DispatchStrategy {
    @Override
    public Courier matchCourier(Order order, List<Courier> couriers) {
        return couriers.stream()
            .min(Comparator.comparingDouble(c ->
                GeoUtils.calculateDistance(order.getRestaurantLocation(),
c.getCurrentLocation())))
            .orElseThrow(() -> new NoCourierAvailableException());
    }
}

// Concrete Strategy B: Multi-Factor Weighted Optimization (Normal Mode)
public class RatingWeightedStrategy implements DispatchStrategy {
    @Override
    public Courier matchCourier(Order order, List<Courier> couriers) {
        return couriers.stream()
            .max(Comparator.comparingDouble(c ->
                (c.getRating() * 0.4) -
(GeoUtils.calculateDistance(order.getRestaurantLocation(), c.getCurrentLocation()) * 0.6)))
            .orElseThrow(() -> new NoCourierAvailableException());
    }
}
```

Pattern 2: State Pattern — Order Lifecycle State Machine

Rather than cluttering code with sprawling `switch-case` branches, the State Pattern encapsulates lifecycle-specific behaviors into dedicated state classes, preventing invalid state jumps:

```
@startuml
interface OrderState <<interface>> {
  + pay(): void
  + accept(): void
  + pickup(): void
  + deliver(): void
  + cancel(): void
}
class PlacedState
class PaidState
class DeliveringState
OrderState <|.. PlacedState
OrderState <|.. PaidState
OrderState <|.. DeliveringState
@enduml
```

Pattern 3: Observer Pattern (Pub-Sub) — Real-Time Event Broadcasting

When an order experiences state transitions (e.g., `OrderPaidEvent`, `FoodReadyEvent`), the Order Service avoids direct coupling with restaurant terminals, courier devices, and notification servers.

Instead, it emits events to Apache Kafka topics, allowing decoupled consumers to react independently.

---

3. Dynamic Behavioral Modeling: Sequence & State Diagrams

3.1 End-to-End Distributed Sequence Diagram

The UML Sequence Diagram captures chronological message flows, synchronous RPC calls, and asynchronous pub-sub events across microservices.

The diagram below details the entire transaction flow from checkout and payment authorization to kitchen confirmation and courier assignment:

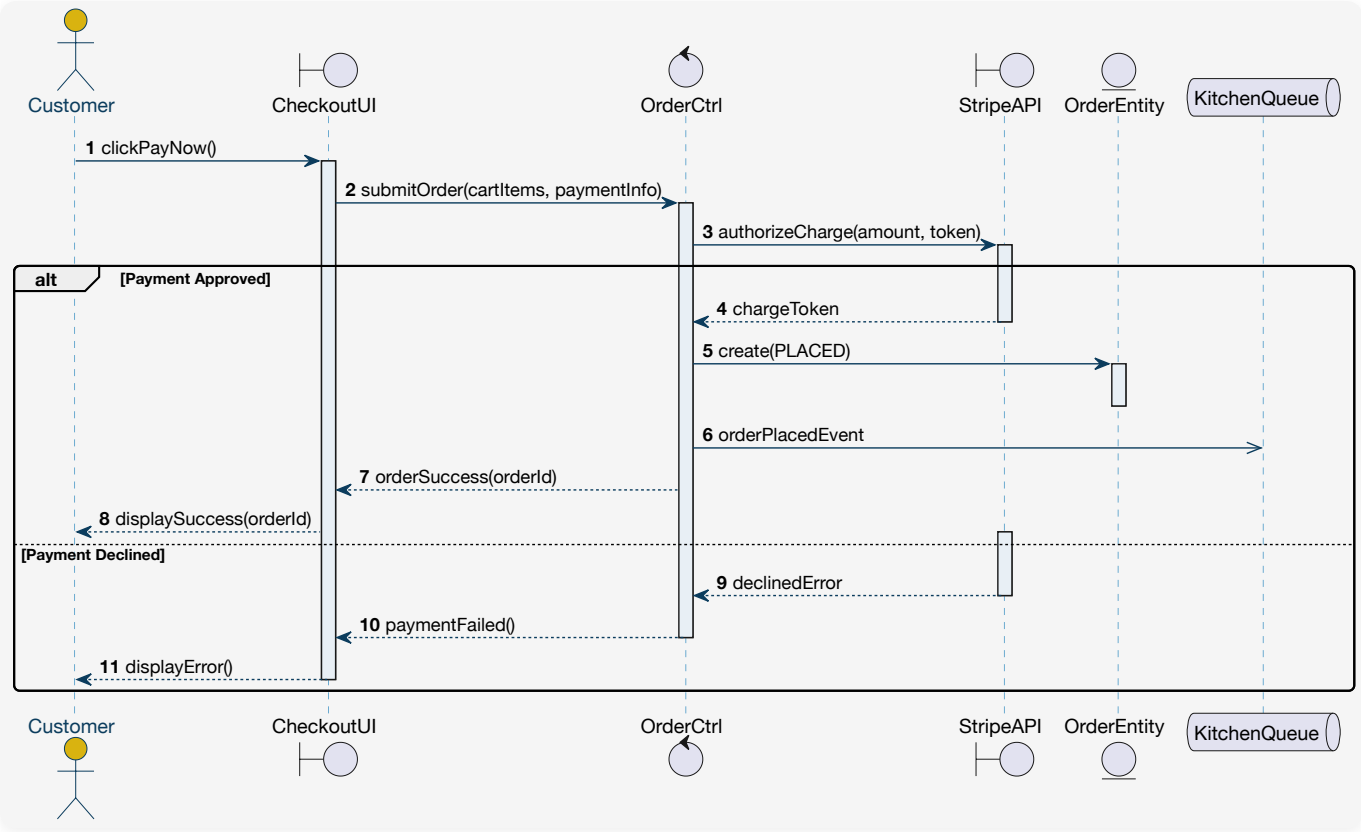


Figure 3.1 QuickBite End-to-End Checkout, Payment & Dispatch Sequence Diagram

Architectural Execution Highlights:

1. **Idempotent Checkout**:
   - The client invokes `POST /orders/checkout` with an `Idempotency-Key`.
   - `Order Service` acquires a Redis distributed lock, rejecting concurrent duplicate clicks.
2. **Synchronous Payment Authorization**:
   - The Payment Service executes a synchronous call to the external Payment Gateway, receiving a unique `charge_id`.

3. **Asynchronous Kafka Event Emission**:
    - Upon successful authorization, `Order Service` emits an `OrderCreatedEvent`.
    - `Restaurant Service` receives the message and triggers an acoustic chime on the kitchen terminal.
4. **Time-Aligned Parallel Dispatch**:
    - The restaurant enters the estimated prep time T_{prep}.
    - `Dispatch Service` schedules courier matching 10 minutes prior to anticipated completion, minimizing driver idle time at the restaurant.

3.2 Order Lifecycle State Machine Diagram

The state machine diagram governs the permissible states, transitions, guard conditions, and side-effect actions of the order entity:

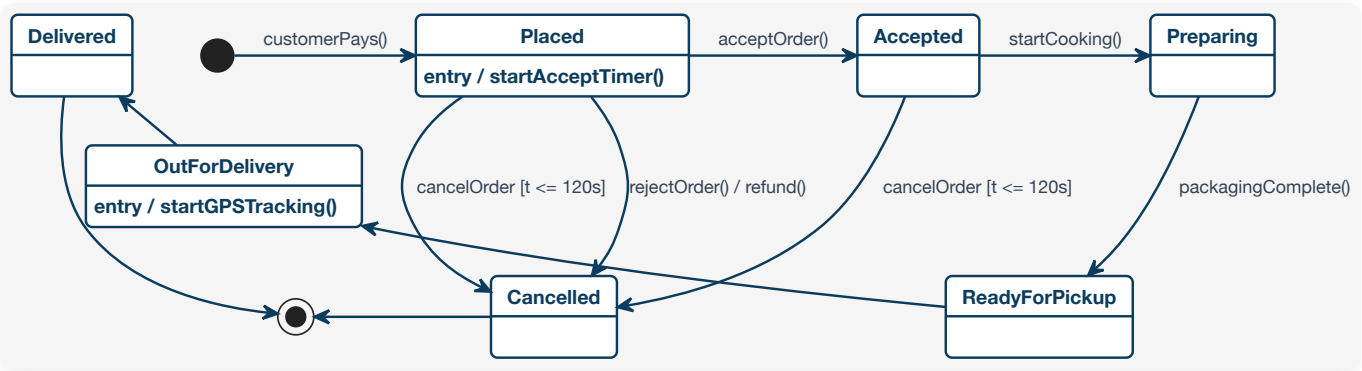


Figure 3.2 QuickBite Order Lifecycle State Machine Diagram

State Transition Matrix:

Source State	Event Trigger	Guard Condition	Target State	Actions & Side Effects
`Placed`	`PAYMENT_SUCCESS`	Gateway authorizes charge	**`Paid`**	Persist payment record; emit event to notify merchant
`Placed`	`TIMEOUT_OR_FAILED`	Unpaid > 15 minutes	**`Cancelled`**	Release inventory hold; close order
`Paid`	`RESTAURANT_ACCEPT`	Kitchen capacity OK	**`Preparing`**	Start kitchen prep timer; schedule dispatch task
`Paid`	`USER_ABORT`	Within 10s grace or unaccepted	**`Refunded`**	Issue 100% automated refund; restore promo code
`Preparing`	`KITCHEN_DONE`	All items ready	**`ReadyForPickup`**	Notify assigned courier: "Order ready for pickup"
`ReadyForPickup`	`COURIER_PICKUP`	Courier 4-digit PIN verified	**`Delivering`**	Open real-time GPS telemetry WebSocket channel
`Delivering`	`COURIER_DELIVERED`	Arrived at location + photo	**`Delivered`**	Issue electronic invoice; prompt for dual-sided review

---

4. Interface & Data Architecture

4.1 Core RESTful API Contracts (OpenAPI 3.0 Summary)

1. Order Checkout Endpoint

- **`POST /api/v1/orders/checkout`**
- **Headers**: `Authorization: Bearer <JWT>`, `Idempotency-Key: <UUID>`
- **Request Body**:

```
{
  "restaurantId": "rest-88392",
  "items": [
    { "menuItemId": "item-101", "quantity": 1, "customizations": ["Extra noodles", "Soft-
boiled egg"] },
    { "menuItemId": "item-205", "quantity": 2, "customizations": ["Less sugar", "Light ice"] }
  ],
  "deliveryAddress": {
    "lat": 24.1790,
    "lng": 120.6482,
    "detail": "No. 100, Wenhua Rd., Xitun Dist., Taichung City (CS Building 3F)"
  },
  "promoCode": "SE2026",
  "paymentMethod": "CREDIT_CARD",
  "paymentToken": "tok_visa_4242"
}
```

- **Response 201 Created**:

```
{
  "orderId": "ord-20261003-9981",
  "status": "Placed",
  "totalAmount": 380,
  "discountAmount": 50,
  "finalAmount": 330,
  "estimatedDeliveryTime": "2026-10-03T12:25:00Z"
}
```

2. Courier Telemetry Endpoint

- **`PUT /api/v1/couriers/me/location`**
- **Headers**: `Authorization: Bearer <Courier-JWT>`
- **Request Body**:

```
{
  "latitude": 24.17924,
  "longitude": 120.64911,
  "speed": 35.5,
  "heading": 85.0,
  "timestamp": 1791000000
}
```

- **Backend Processing Action**:
  - Redis Geo command: `GEOADD couriers:online:geo 120.64911 24.17924 courier-402`
  - If courier has active delivery, publish to Redis Pub/Sub: `PUBLISH order:ord-9981:tracking "{...}"`

4.2 Polyglot Persistence Architecture

Different domains require divergent consistency and latency characteristics. QuickBite adopts specialized data engines:

Storage Engine	Entities & Domain Data	Architectural Rationale
PostgreSQL (RDBMS)	Accounts, Orders, Order Items, Ledger Entries	Strict ACID compliance; foreign key referential integrity; financial auditability.
Redis Cluster (In-Memory)	User sessions, active carts, courier GeoHash indexing, distributed idempotency locks (`Redlock`)	Sub-millisecond latency; native geospatial distance calculation commands (`GEORADIUS`).
Apache Kafka (Event Stream)	Global event log (`order.*` , `courier.*` , `payment.*`)	High-throughput buffering; traffic surge smoothing; enables Event Sourcing and audit replay.
Amazon S3 / MinIO	Restaurant menu imagery, delivery confirmation photos	Highly durable, cost-effective object storage distributed via CloudFront CDN.

---

5. Design Principles & Architectural Trade-offs

5.1 SOLID Principles Compliance Matrix (Aligned with Slide 05)

1. **Single Responsibility Principle (SRP)**:
   - `OrderCalculator` focuses exclusively on financial arithmetic; `OrderRepository` handles database persistence; `DispatchEngine` manages courier matching.
2. **Open/Closed Principle (OCP)**:
   - Introducing a new payment gateway requires implementing the `PaymentGateway` interface without altering the existing `CheckoutService` implementation.
3. **Liskov Substitution Principle (LSP)**:
   - Anywhere an abstract `User` is expected, derived subtypes (`Customer` , `Courier`) can be substituted without invalidating system contracts.
4. **Interface Segregation Principle (ISP)**:
   - Rather than monolithic interfaces, operations are segregated into `MenuManageable` , `OrderFulfillable` , and `DeliveryFulfillable` .
5. **Dependency Inversion Principle (DIP)**:
   - High-level business logic depends upon domain abstractions (e.g., `NotificationSender`), with concrete implementations (e.g., `TwilioSender` , `SendGridSender`) injected at runtime via Spring IoC.

5.2 CAP Theorem Trade-off Strategy

Per the CAP Theorem, distributed architectures must balance Consistency (C) and Availability (A) during network partitions (P):

Distributed CAP Strategy	CP Subsystem: Orders & Financial Ledger	AP Subsystem: GPS Telemetry & Reviews
Primary Architectural Priority	Strong Consistency (CP)	High Availability (AP)
Concurrency & Locking	Strict distributed locks (PostgreSQL ACID / Redlock)	Eventual Consistency accepted
Partition Fallback Behavior	Rejects order if consensus cannot be reached (zero double-charging)	Displays last-known cached GPS coordinates during network jitter

---

Authored by: Feng Chia University Software Engineering Teaching Team (Prof. Nien-Lin Hsueh)
Companion Document: QuickBite Software Requirements Specification (SRS)